Cliente de un servicio WCF en una solución implementada con Clean Architecture en .NET 6

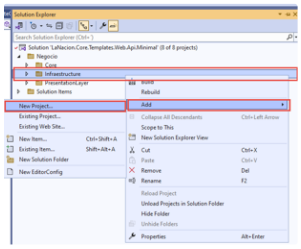
Este documento muestra los pasos necesarios para crear un cliente de un servicio WCF en una solución implementada con Clean Architecture en .NET 6, como es el caso del template de minimal api.

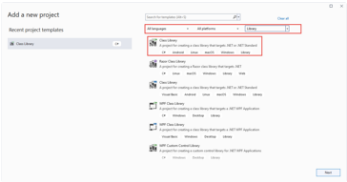
> Para instalar un template con un cliente SOAP funcionando
> `dotnet new install lanacion.Core.MinimalWebApiConClienteWCF.Template --nuget-source https://pkgs.dev.azure.com/lndigital/_packaging/lanacion-v2/nuget/v3/index.json --interactive`

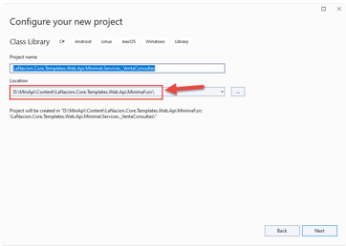
Clean Architecture es una forma de organizar una solución de software organizándola en capas que incluyen Application, Domain, Infrastructure y Presentation. El cliente WCF se implementará en la capa de Infrastructure y se utilizará en la capa de Application. Aquí te dejo una guía paso a paso

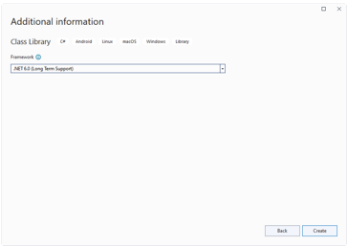
Paso 1: Crear el cliente WCF en la capa de Infrastructure

1. Crear un proyecto de tipo class library .Net 6 en la capa de Infraestructura









Agregarle al nuevo proyecto runa referencia al proyecto *LaNacion.Core.Templates.Web.Api.Minimal.Application.Interfaces* de la solución creda con el template de minimal api.

2. **Agregar un proxy de servicio WCF**: En .NET Core y .NET 6, puedes utilizar la herramienta `dotnet-svcutil` para generar un proxy de servicio WCF.

```
dotnet tool install --global dotnet-svcutil
dotnet-svcutil https://lnws.lanacion.com.ar/Ventas/VentaConsultasService.svc?Wsdl -pf LaNacion.Core.Templates.Web.Api.Minimal.Services.Vent
```

Esto generará el proxy de servicio en un archivo `.cs` debe incluirse en el proyecto de Infrastructure especificado.

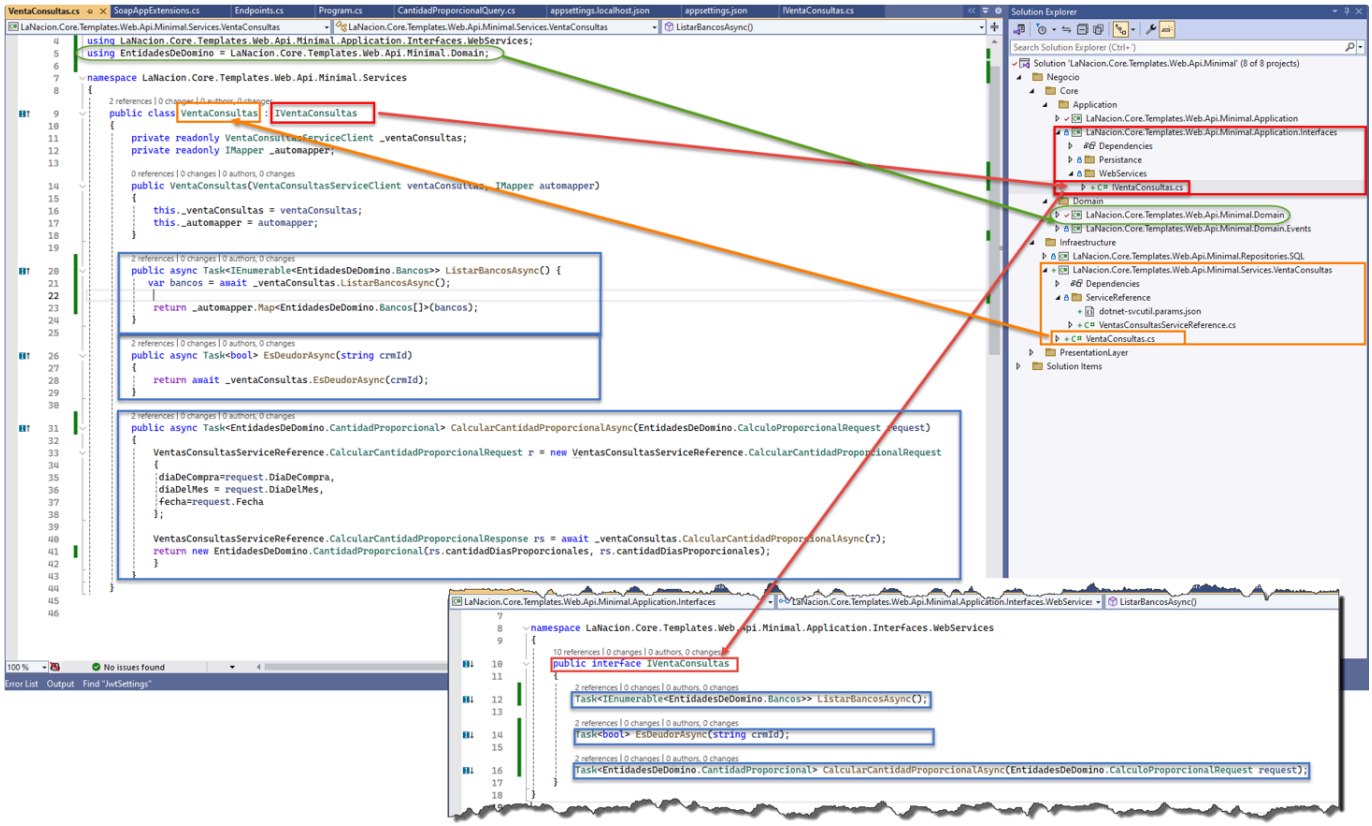
3. **Agregar el archivo proxy generado a tu proyecto de Infrastructure**: Incluir el archivo `.cs` generado por `dotnet-svcutil` en el proyecto de Infrastructure creado, en el ejemplo `LaNacion.Core.Templates.Web.Api.Minimal.Services.VentaConsultas.csproj`.

   Si se usó el argumento `-pf [nombreDelProyecto.csprj]` el proxy .cs se agrega automáticamente al proyecto.

4. **Crear una clase de cliente para encapsular la lógica de comunicación con el servicio WCF**: La idea con esta clase es que oficie de “mediador” o “adapter” entre la capa de aplicación y el proxy creado en el punto anterior de manera de abstraer la complejidad y ofrecer un lugar donde mapear las entidades de negocio a los contratos de servicios. Las capas de presentación y Aplicación deberían

mantenerse abstractas a los data-contract de los servicios. Este Wrapper por su puesto debe tener su correspondiente interface en la capa de Interfaces. De esta manera, exponiendo únicamente la funcionaidad requerida para los caso de negocio q estamos implementando, mantenemos el principio de Interface Segregada de SOLID.

En el ejemplo, *VentaConsultas*, el “adapter”, recibe por inyección de dependencias una referencia a *VentaConsultasServiceClient* que es el proxy al servicio creado en el punto 2.

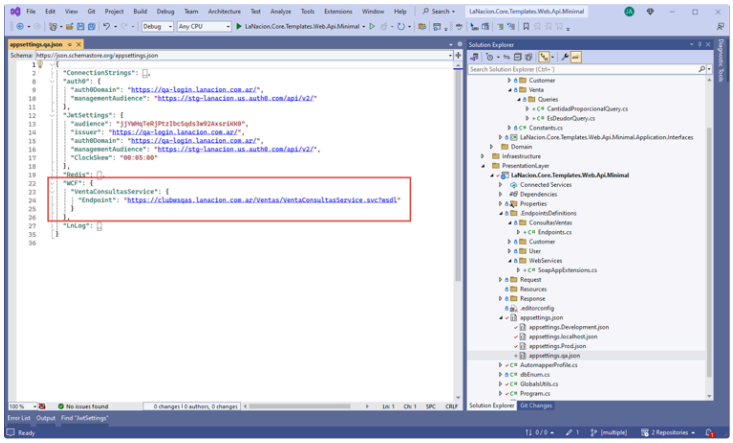


Paso 2: Configurar la inyección de dependencias

1. **Registrar el cliente WCF en el contenedor de DI**: En el proyecto de Presentación, crear una clase de configuración para registrar el cliente WCF.

```
1  using LaNacion.Core.Templates.Web.Api.Minimal.Application.Interfaces.WebServices;
2  using LaNacion.Core.Templates.Web.Api.Minimal.Services;
3  using System.ServiceModel;
4  using VentasConsultasServiceReference;
5  using static Humanizer.In;
6
7  namespace LaNacion.Core.Templates.Web.Api.Minimal.EndpointsDefinitions.WebServices
8  {
9      public static class SoapAppExtensions
10     {
11         public static IServiceCollection AddSoapServices(this IServiceCollection services, IConfiguration configuration)
12         {
13             services.AddScoped<IVentaConsultas, VentaConsultas>();
14             services.AddScoped<VentaConsultasServiceClient>(provider =>
15             {
16                 var endpoint = new EndpointAddress(configuration["WCF:VentaConsultasService:Endpoint"]);
17                 var binding = new BasicHttpBinding();
18                 binding.Name = "VentaConsultasServiceSoap";
19                 binding.Security.Mode = BasicHttpSecurityMode.Transport;
20
21                 return new VentaConsultasServiceClient(binding, endpoint);
22             });
23
24             return services;
25         }
26     }
27 }
```

La url del servicio se configura en los appSettings. En los casos q el servicio funcione sobre http**s**, debe ajustarse el modo de seguridad a Transporte (línea 19)



2. **Registrar los servicios de Infrastructure en la capa Presentation**: En tu proyecto Presentation (por ejemplo, en `Program.cs` o `Startup.cs`), llama al método de extensión para registrar los servicios de Infrastructure.

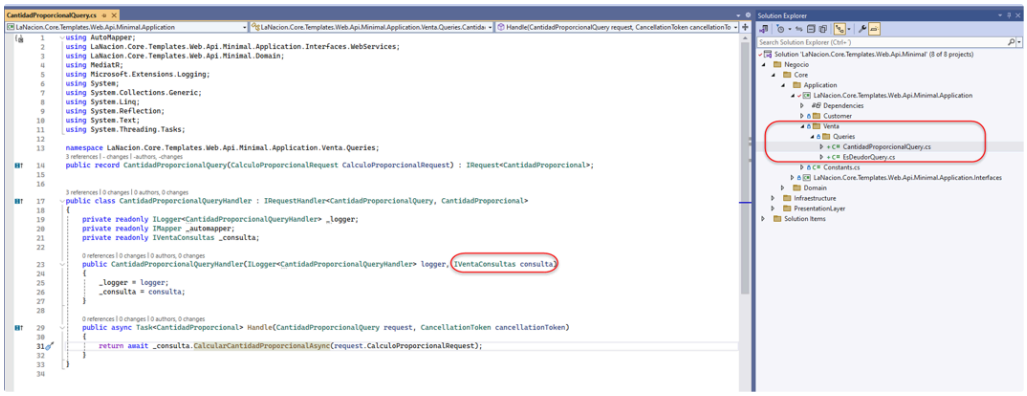
```
//Servcios
builder.Services.AddSoapServices(builder.Configuration);
```

3. **Configurar las URLs de los servicios según ambiente**:

```
  "WCF": {
    "VentaConsultasService": {
      "Endpoint": "https://clubwsqas.lanacion.com.ar/Ventas/VentaConsultasService.svc?wsdl"
    },
    "OTROService": {
      "Endpoint": "https://clubwsqas.lanacion.com.ar/UnServicio/otroService.svc?wsdl"
    }
  },
```

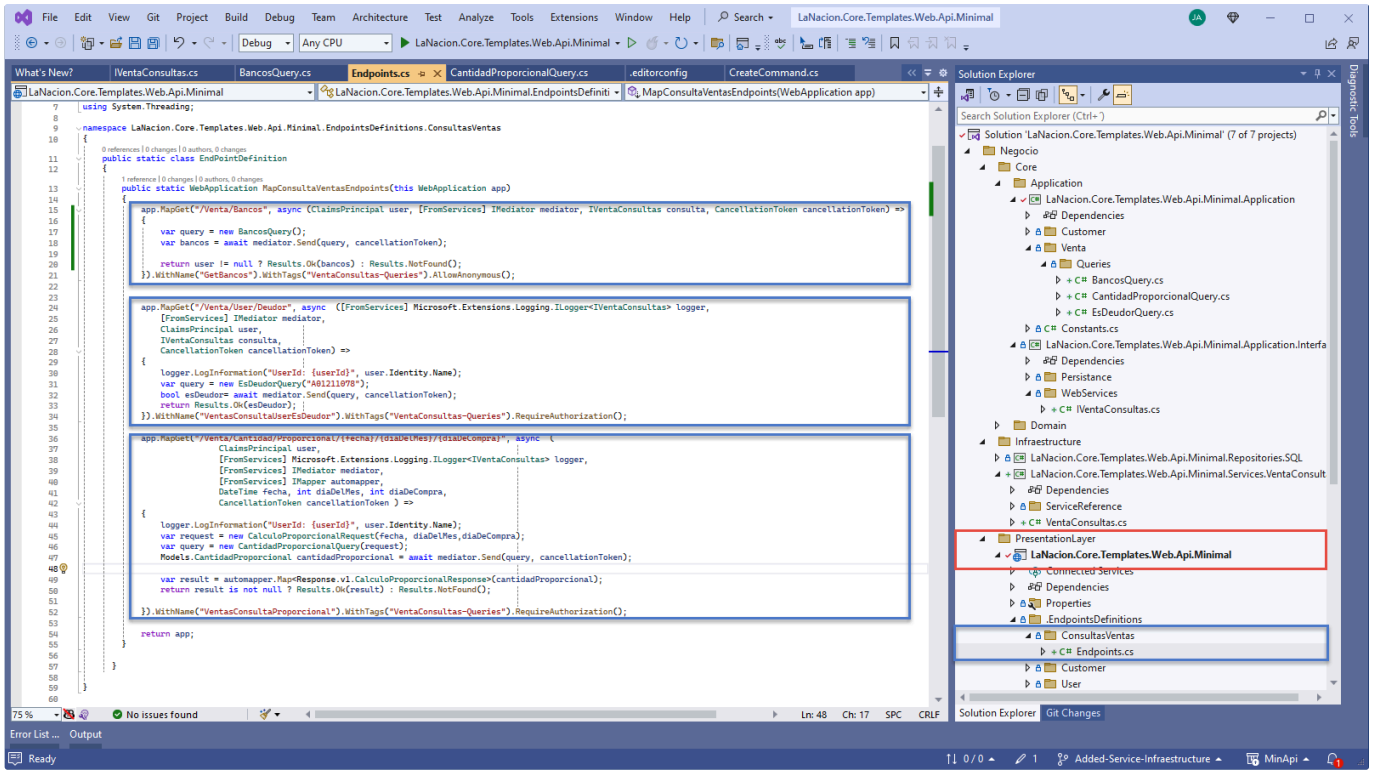
Paso 3: Usar los servicios

El uso de los servicios debe realizarse dentro del marco de CQRS. (ver el video Recorremos el template de Solución de Web APIs-20220810_110436-Grabación de la reunión.mp4 a partir del minuto 35)



En el ejemplo se crearon 2 queries, aunque el proceso para comandos es el mismo. La idea general es que tanto los queries como los comandos, no interactúen directamente con el proxy, sino con el “adapter”. Para esto recibe una referencia a la interface del adapter y este es inyectado por DI.

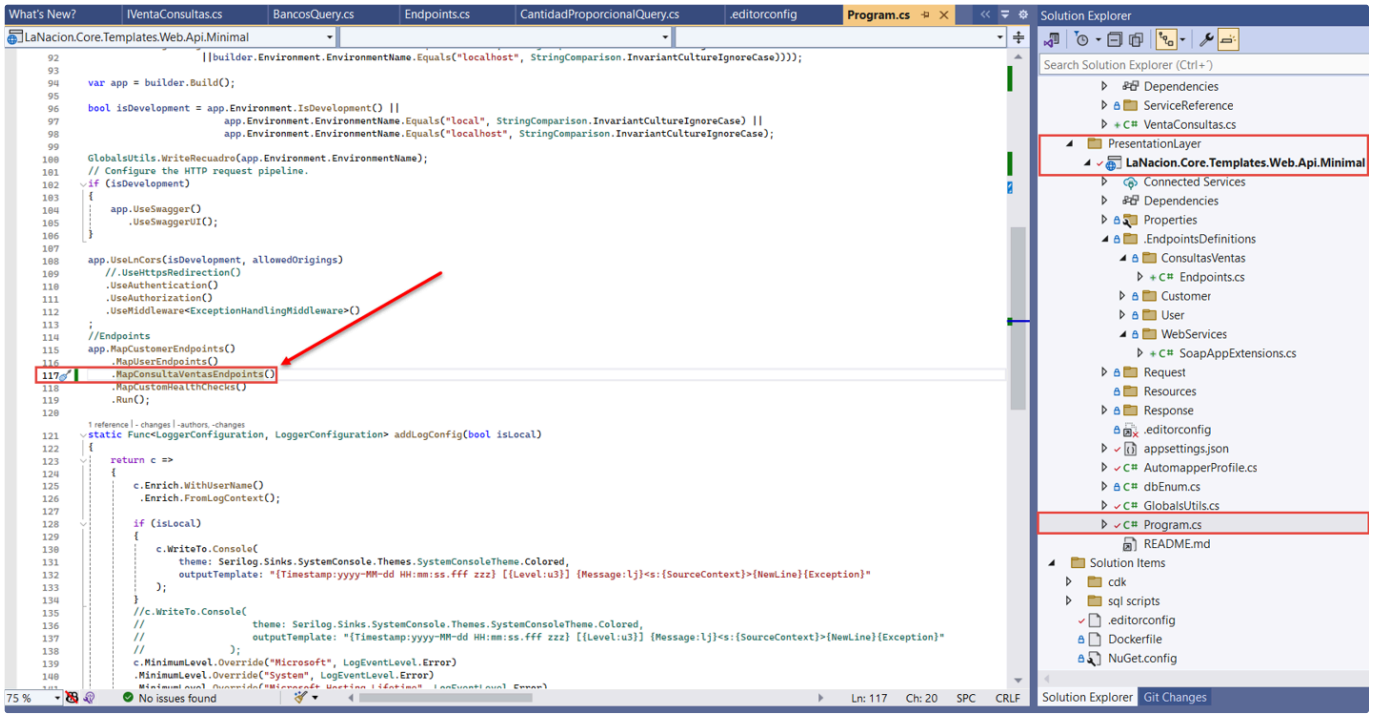
Y estas queries (o commandos) se llaman usando MediatR desde los endpoints de la API, que se definen de esta manera:




```
namespace LaNacion.Core.Templates.Web.Api.Minimal.EndpointsDefinitions.ConsultasVentas
{
    public static class EndPointDefinition
    {
        public static WebApplication MapConsultaVentasEndpoints(this WebApplication app)
        {
            app.MapGet("/Venta/Bancos", async (ClaimsPrincipal user, [FromServices] IMediator mediator, IVentaConsultas consulta, Cancella
            {
                var query = new BancosQuery();
                var bancos = await mediator.Send(query, cancellationToken);

                return user != null ? Results.Ok(bancos) : Results.NotFound();
            }).WithName("GetBancos").WithTags("VentaConsultas-Queries").AllowAnonymous();


            app.MapGet("/Venta/User/Deudor", async  ([FromServices] Microsoft.Extensions.Logging.ILogger<IVentaConsultas> logger,
                [FromServices] IMediator mediator,
                ClaimsPrincipal user,
                IVentaConsultas consulta,
                CancellationToken cancellationToken) =>
            {
                logger.LogInformation("UserId: {userId}", user.Identity.Name);
                var query = new EsDeudorQuery("A01211078");
                bool esDeudor= await mediator.Send(query, cancellationToken);
                return Results.Ok(esDeudor);
            }).WithName("VentasConsultaUserEsDeudor").WithTags("VentaConsultas-Queries").RequireAuthorization();

            app.MapGet("/Venta/Cantidad/Proporcional/{fecha}/{diaDelMes}/{diaDeCompra}", async  (
                            ClaimsPrincipal user,
                            [FromServices] Microsoft.Extensions.Logging.ILogger<IVentaConsultas> logger,
                            [FromServices] IMediator mediator,
                            [FromServices] IMapper automapper,
                            DateTime fecha, int diaDelMes, int diaDeCompra,
                            CancellationToken cancellationToken ) =>
            {
                logger.LogInformation("UserId: {userId}", user.Identity.Name);
                var request = new CalculoProporcionalRequest(fecha, diaDelMes,diaDeCompra);
                var query = new CantidadProporcionalQuery(request);
                Models.CantidadProporcional cantidadProporcional = await mediator.Send(query, cancellationToken);

                var result = automapper.Map<Response.v1.CalculoProporcionalResponse>(cantidadProporcional);
                return result is not null ? Results.Ok(result) : Results.NotFound();
            }).WithName("VentasConsultaProporcional").WithTags("VentaConsultas-Queries").RequireAuthorization();
            return app;
        }
    }
}
```

En este caso se declaran 3 endpoints que utilizando MediatR llaman a las correspondientes Queries. En este caso 1 (`"/Venta/Bancos"`) endpoint puede ser consultado sin necesidad de autenticarse (`.AllowAnonymous()`) y los otros 2 (`"/Venta/User/Deudor"` , `"/Venta/Cantidad/Proporcional/{fecha}/{diaDelMes}/{diaDeCompra}"`) requieren que el usuario se haya autenticado previamente (`.RequireAuthorization()`).

Una vez definidos los endpoints hay que referenciarlos para que la aplicación sea consciente de su existencia:



Paso 4: Registrar los servicios de autenticación

El template tiene la posibilidad de usar autenticación con Auth0 y custom con algoritmos simétricos y asimétricos (usando clave pública en formato PEM) para la verificación de la firma. El template contiene un ejemplo de cada tipo, Auth0 y Custom, así como ejemplos de como utilizar políticas de autorización. De esté código eliminar lo que no se utilice. Para Auth0 por ejemplo:

Para Auth0 es necesario configurar las siguientes lave en el appsettings.json. Los valores productivos deben especificarse en el pipeline de deploy como variables del release. La Key `"useRedis"` especifica si las claves públicas para validar la firma del token se cachen en Redis. En caso de ser `"false"` , las claves públicas no se cacheará en Redis y por tanto por cada validación de un Token se realizará un request a Auth0 para obtener la información necesaria.

```
  "auth0": {
    "useRedis": "true",
    "auth0Domain": "https://qa-login.lanacion.com.ar/",
    "managementAudience": "https://stg-lanacion.us.auth0.com/api/v2/"
  },
  "JwtSettings": {
    "audience": "jjYWHqTeRjPtzIbcSqds3w92AxsriKK0",
    "issuer": "https://qa-login.lanacion.com.ar/",
    "auth0Domain": "https://qa-login.lanacion.com.ar/",
    "managementAudience": "https://stg-lanacion.us.auth0.com/api/v2/",
    "ClockSkew": "00:05:00"
```

```
12  },
13  "Redis": {
14    "Configuration": "srviispro01.lanacion.com.ar:6379"
15  },
```